

Core Concepts and Architecture (Current Standard)

Modern Spring Security relies on a few fundamental concepts and architectural patterns that align with today's cloud-native and microservice architectures:

- **The Security Filter Chain:** Spring Security operates as a series of Servlet Filters intercepting HTTP requests. The `DelegatingFilterProxy` hands the request to a `FilterChainProxy`, which manages a list of `SecurityFilterChain` beans tailored to specific security requirements (e.g., one chain for API keys, another for basic login).
- **Lambda DSL Configuration:** Legacy configuration styles have been removed. Today, security rules are written using a fluent, Lambda-based Domain Specific Language (DSL). You define rules by injecting a `SecurityFilterChain` bean rather than extending adapter classes.
- **OAuth 2.0 and OpenID Connect (OIDC):** Instead of storing usernames and passwords in a local database, modern applications delegate authentication to external Identity Providers (IdPs) like Okta, Auth0, Keycloak, or Google. Spring Security acts as an OAuth2 Client or Resource Server.
- **JWT (JSON Web Tokens):** For stateless, scalable microservices, JWTs are the market standard. Spring Security provides built-in Resource Server configurations to validate JWT signatures and extract claims (like user roles) without needing to query a database on every request.
- **Method-Level Security:** Instead of just securing URLs, modern architectures use annotations like `@PreAuthorize` and `@PostAuthorize` directly on Java methods. This allows for granular Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) deeply embedded in the business logic.